

BIKE_MLThre: A Machine-Learning-Assisted Threshold-Selection Variant of BIKE

Algorithm Specification

This document describes BIKE_MLThre, a code-based key encapsulation mechanism that introduces a learning-assisted residual threshold selection strategy into the bit-flipping decapsulation procedure. The main innovation of BIKE_MLThre lies in replacing the static threshold rule used during decoding with an adaptive threshold control mechanism that exploits residual syndrome information, iteration-dependent decoding states, and learned decision patterns to select more suitable flipping thresholds during each decoding round.

By modeling threshold selection as an internal decoding-control problem, BIKE_MLThre aims to improve the reliability of the decoder and reduce the decoder failure rate (DFR), particularly in cases where a fixed hand-crafted threshold may be suboptimal for the current syndrome residual distribution. The learning-assisted selector is embedded directly into the decapsulation process and operates on decoder-side information only, thereby providing adaptive decoding behavior without introducing an additional communication step.

1 Algorithm Description

1.1 Construction overview

BIKE_MLThre introduces an adaptive threshold-control architecture into the decapsulation decoder. The core design is a residual threshold selector that dynamically adjusts the flipping threshold used by the BGF decoder according to the current decoding state. Instead of relying on a predetermined affine threshold rule throughout all decoding iterations, BIKE_MLThre computes an iteration-specific correction term and applies it to the baseline decoding threshold, thereby enabling state-dependent threshold refinement during decapsulation.

During each decoding iteration, the decoder extracts a compact feature vector from internal decoding observables, including the current syndrome weight, the iteration index, the cardinalities of the black and gray sets, and lightweight statistics derived from the counter distribution. These quantities characterize the residual error pattern and the progress of the bit-flipping process. Let this feature vector be denoted by x_i at iteration i . A trained low-cost inference module f_θ maps x_i to

a threshold correction value ΔT_i , which is then combined with the reference BGF threshold to obtain the effective decoding threshold:

$$T_i = \text{clip}(T_{\text{BGF}}(S_i, i) + \Delta T_i; T_{\min}; T_{\max})$$

This residual formulation allows the learned module to perform local threshold refinement rather than full threshold replacement. The clipping operation constrains the adjusted threshold within an admissible range, preventing unstable threshold deviations. In addition, an optional confidence gate may suppress the learned correction by setting $\Delta T_i = 0$ when the model confidence is below a predefined threshold. This design provides a controlled adaptive decoding mechanism: the learned selector influences the decoder only when its prediction is sufficiently reliable, while uncertain cases are handled by the reference threshold rule.

1.2 Algebraic Setting and Parameter Instances

Let F_2 be the binary field and let $R = F_2[X]/(X^r - 1)$. The private-key space is

$$H_w = \{(h_0, h_1) \in R^2 : |h_0| = |h_1| = w/2\}$$

and the error space is

$$E_t = \{(e_0, e_1) \in R^2 : |e_0| + |e_1| = t\}.$$

BIKE_MLThre provides three parameter categories corresponding to 128-bit, 256-bit, and 512-bit target security levels. The shared-secret length ℓ is not fixed uniformly across all categories, but is chosen to match the intended security profile. In the 128-bit and 256-bit instances, $\ell = 256$ bits is used because a 256-bit shared secret already provides sufficient symmetric-key material for both categories and avoids unnecessary output expansion. In the 512-bit instance, $\ell = 512$ bits is used to align the derived shared-secret length with the higher security target and to prevent the KDF output length from becoming a bottleneck in the overall security claim. Thus, the algebraic parameter set determines the code-based hardness level, while ℓ determines the length of the derived shared secret for each security category.

The concrete parameter choices for the three BIKE_MLThre security categories are summarized in Table 1. Each instance is specified by the ring parameter r , the secret weight w , the error weight t , the shared-secret length ℓ , and the target decoder failure rate.

Table 1.Parameter Categories and Target DFRs of BIKE_MLThre

Instance	Security target	r	w	t	Target DFR
BIKE_M LThre-128	128 bit	12,323	142	134	$\leq 2^{-128}$
BIKE_M LThre-256	256 bit	40,973	274	264	$\leq 2^{-256}$
BIKE_M LThre-512	512 bit	150001	546	524	$\leq 2^{-512}$

For each parameter category, BIKE_MLThre uses an affine reference threshold function as the baseline component of the residual threshold-control mechanism. Let $s_i = wt(S_i)$ denote the syndrome weight at decoding iteration i . The reference threshold functions are defined as follows:

Table 2.Reference Threshold Functions for BIKE_MLThre Instances

Instance	Reference threshold function
BIKE_MLThre-128	$T(S) = \max(\lfloor 0.0069722 * S + 13.530 \rfloor, 36)$
BIKE_MLThre-256	$T(S) = \max(\lfloor 0.0052650 * S + 15.2588 \rfloor, 52)$
BIKE_MLThre-512	$T(S) = \max(\lfloor 0.0001734 * S + 136.5610 \rfloor, 138)$

These reference functions provide the deterministic threshold component used by the BGF decoder. In BIKE_MLThre, the final threshold applied at iteration (i) is obtained by adding a learned residual correction term to the corresponding reference threshold and then constraining the result to an admissible threshold interval. Thus, the affine function defines the reference decoding behavior, while the learning-assisted module provides instance- and iteration-dependent threshold refinement.

1.3 KEM algorithms

BIKE_MLThre defines a key encapsulation mechanism consisting of three algorithms:

$$KeyGen, \quad Encaps, \quad Decaps.$$

For a security category, $\lambda \in 128, 256, 512$, let $(r_\lambda, w_\lambda, t_\lambda)$ denote the corresponding code-based parameter set. Let (μ_λ) denote the seed length used for error sampling and ciphertext masking, and let (ℓ_λ) denote the output length of the derived shared secret. In the reference profile, $\ell_{128}=256, \ell_{256}=256, \ell_{512}=512$.

The functions used by the KEM are defined as follows. The constant-weight error sampler is

$$H_{\text{err}}: \{0,1\}^{\mu_\lambda} \rightarrow E_{t_\lambda},$$

the masking function is

$$L: E_{t_\lambda} \rightarrow \{0,1\}^{\mu_\lambda},$$

and the key-derivation function is

$$KDF: \{0,1\}^{\mu_\lambda} \times R_{r_\lambda} \times \{0,1\}^{\mu_\lambda} \rightarrow \{0,1\}^{\ell_\lambda}.$$

Key Generation

$\text{KeyGen}_\lambda()$:

1. Sample $(h_0, h_1) \leftarrow \mathcal{H}_{w_\lambda}$.
2. If h_0 is not invertible in (R_{r_λ}) , resample h_0, h_1 .
3. Compute $h \leftarrow h_1 h_0^{-1} \in R_{r_\lambda}$.
4. Sample a secret fallback value $\sigma \leftarrow \{0,1\}^{\mu_\lambda}$.
5. Output $sk=(h_0, h_1, \sigma), pk=h$.

Encapsulation

$Encaps_{\lambda}(pk=h)$:

1. Sample a random seed $m \leftarrow \{0,1\}^{\mu_{\lambda}}$.
2. Derive a constant-weight error vector $(e_0, e_1) \leftarrow H_{\text{err}}(m), (e_0, e_1) \in E_{t_{\lambda}}$.
3. Compute $c_0 \leftarrow e_0 + e_1 h \in R_{r_{\lambda}}$.
4. Compute $c_1 \leftarrow m \oplus L(e_0, e_1)$.
5. Derive the shared secret $K \leftarrow KDF(m, c_0, c_1), K \in \{0,1\}^{\ell_{\lambda}}$.
6. Output $c = (c_0, c_1), K$.

Decapsulation

$Decaps_{\lambda}(sk=(h_0, h_1, \sigma), c=(c_0, c_1))$:

1. Compute the decoding syndrome $s \leftarrow c_0 h_0 \in R_{r_{\lambda}}$.
2. Run the BIKE_MLThre adaptive decoder:

$$e' = (e'_0, e'_1) \leftarrow \text{BIKEMLThre.Decoder}_{\theta}(s, h_0, h_1, \lambda).$$

If decoding fails, set

$$e' \leftarrow (0, 0).$$

3. Recover the candidate seed

$$m' \leftarrow c_1 \oplus L(e').$$

4. Recompute the candidate error vector

$$e^{\star} \leftarrow H_{\text{err}}(m').$$

5. Define the verification bit

$$b = \begin{cases} 1, & \text{if } (e' = e^{\star}) \\ 0, & \text{otherwise} \end{cases}$$

6. Derive the shared secret by constant-time selection:

$$K \leftarrow \{KDF(m', c_0, c_1), \text{if}(b=1); KDF(\sigma, c_0, c_1), \text{if}(b=0)\}$$

The selection between the two branches shall be implemented without secret-dependent timing or branching.

7. Output

$$K \in \{0,1\}^{\ell_\lambda}.$$

In this specification, *Herr* is instantiated as a constant-weight sampler seeded by (μ_λ) bits. The functions (L) and (KDF) are instantiated using hash-based components such as SHA-384 and SHAKE256 according to the selected implementation profile. The output length of (KDF) is determined by (ℓ_λ) : the 128-bit and 256-bit instances output 256-bit shared secrets, while the 512-bit instance outputs a 512-bit shared secret.

1.4 Learning-assisted decoder

The BIKE_MLThre decoder is a BGF-derived iterative decoder equipped with a residual threshold-control mechanism. Its main modification is the computation of the bit-flipping threshold at each decoding iteration. Instead of applying a fully predetermined threshold sequence, the decoder derives an iteration-dependent correction term from the current residual decoding state and applies this correction to the reference threshold.

Let s_i denote the syndrome at iteration i , and let

$$S_i = \text{wt}(s_i)$$

be the corresponding syndrome weight. At each iteration, the decoder extracts an 11-dimensional feature vector x_i from the internal decoding state. The feature vector is defined as

$$x_i = (\text{level}, i, S_{\text{init}}, S_{\text{prev}, i}, S_i, \Delta S_i, z_{0,i}, z_{1,i}, T_{\text{var}, i}, T_{\text{lb}}, T_{\text{base}, i}).$$

The meaning of each feature is summarized in Table 3.

Table 3. Description of the 11-dimensional input features of the residual threshold selector.

Feature	Description
<i>level</i>	Security-level identifier. The values 128, 256, and 512 correspond to the 128-bit, 256-bit, and 512-

	bit security categories, respectively.
i	Current decoding iteration index.
S_{init}	Initial syndrome weight at the beginning of decoding.
$S_{\text{prev},i}$	Syndrome weight in the previous decoding iteration.
S_i	Current syndrome weight at iteration i , defined as $S_i = wt(s_i)$.
ΔS_i	Decrease in syndrome weight from the previous iteration to the current iteration, defined as $\Delta S_i = \max(S_{\text{prev},i} - S_i, 0)$. It reflects whether the decoder is making progress.
$z_{0,i}$	Number of newly appeared one positions in the current syndrome compared with the previous syndrome. It describes newly introduced unsatisfied parity checks.
$z_{1,i}$	Number of one positions in the current syndrome that overlap with the previous syndrome. It describes the persistent part of the residual syndrome.
$T_{\text{var},i}$	Variable threshold value computed by the baseline threshold function under the current syndrome weight.
T_{lb}	Lower-bound or initial-state-related reference threshold. In the 512-bit dynamic-threshold implementation, it may be computed from S_{init} .
$T_{\text{base},i}$	Baseline threshold actually used at iteration i before ML-based residual correction.

The syndrome-weight decrease is computed as

$$\Delta S_i = \max(S_{\text{prev},i} - S_i, 0).$$

This quantity reflects the convergence behavior of the decoder. A larger (ΔS_i) indicates that the current iteration effectively reduces the residual syndrome weight, whereas a value close to zero indicates weak progress or possible stagnation.

The features $z_{0,i}$ and $z_{1,i}$ describe the transition pattern between two consecutive syndromes. Specifically, $z_{1,i}$ counts the one positions that remain active in both the previous and current syndromes, while $z_{0,i}$ counts the one positions newly appearing in the current syndrome. They can be expressed as

$$|supp(s_i) \cap supp(s_{i-1})|,$$

and

$$|supp(s_i) \setminus supp(s_{i-1})|.$$

These two features provide information about whether the bit-flipping process is converging, oscillating, or introducing new unsatisfied parity checks.

These two features provide information about whether the bit-flipping process is converging, oscillating, or introducing new unsatisfied parity checks.

The trained residual threshold selector f_θ maps the feature vector x_i to a classification output:

$$z_i = f_\theta(x_i).$$

The output semantics are the same for all security categories: the model predicts an integer threshold correction ΔT_i , which is added to the baseline threshold. Let D_{sec} denote the candidate correction set associated with the selected security category. The residual correction is selected as

$$D_{sec} \left[\arg \max_k z_{i,k} \right]$$

The output dimension of f_θ therefore depends on the selected security category:

$$|D_{sec}|.$$

The security-category-specific candidate correction sets are summarized in Table 4.

Table 4. Security-category-specific candidate correction sets.

Security category	Candidate correction set
128-bit	$D_{128}=\{-7,-6,-5,-4,-3,-2,-1,0\}$
256-bit	$D_{256}=\{-10,-9,-8,-7,-6,-5,-4,-3,-2,-1,0\}$
512-bit	$D_{512}=-2,-1,0,1,2$

The effective threshold used by the BGF decoder at iteration (i) is then computed as

$$T_i = \text{clip}(T_{\text{base},i} + \Delta T_i, T_{\min}, T_{\max}).$$

Here, $T_{\min}$ and $T_{\max}$ define the admissible threshold interval. The clipping operation prevents unstable threshold deviations and ensures that the learned correction remains within a controlled range.

The adaptive decoder is specified as follows.

BIKEMLThre.Decoder $_{\theta}(s, h_0, h_1, \lambda)$:

Input: syndrome (s), sparse parity polynomials (h_0, h_1), and parameter category $\lambda \in 128, 256, 512$.

Output: an error estimate (e'), or the failure symbol ($\perp$).

1. Initialize $e \leftarrow 0, s_1 \leftarrow s, i \leftarrow 1$.
2. For the prescribed number of decoding iterations, repeat the following steps.
 - a. Compute all UPC counters (ctr_j) from the current syndrome (s_i) and the sparse parity-check structure.
 - b. Compute the syndrome weight $S_i = \text{wt}(s_i)$.
 - c. Construct the feature vector

$$x_i = (\text{level}, i, S_{\text{init}}, S_{\text{prev},i}, S_i, \Delta S_i, z_{0,i}, z_{1,i}, T_{\text{var},i}, T_{\text{lb}}, T_{\text{base},i})$$

d. Evaluate the trained residual threshold selector

$$z_i = f_\theta(x_i),$$

e. Select the residual threshold correction

$$\Delta T_i = D_{sec} \left[\arg \max_k z_{i,k} \right].$$

f. Compute the effective threshold $T_i = \text{clip}(T_{\text{base},i} + \Delta T_i, T_{\min}, T_{\max})$.

g. Apply the BGF flipping rule using threshold T_i . That is, flip each candidate position whose UPC counter satisfies the threshold condition determined by T_i , and update the error estimate e and syndrome (s_i accordingly.

3. After the fixed number of iterations, output $e' = e$. If the final syndrome satisfies $s_{\text{final}} = 0$. Otherwise, output the failure symbol $\perp$. A recommended reference instantiation of f_θ is a small quantized multilayer perceptron with 11 input features, one hidden layer of width 16, and $|D_{sec}|$ output classes corresponding to the selected security category. Equivalently,

$$f_\theta: \mathbb{R}^{11} \rightarrow \mathbb{R}^{|D_{sec}|}.$$

The inference module should be implemented using fixed-size integer arithmetic and a fixed execution schedule. The model parameters, quantization format, feature normalization constants, security-category-specific label set D_{sec} , and admissible threshold interval ($T_{\min}, T_{\max}$

) shall be treated as part of the algorithm instance.

1.5 Offline training procedure

BIKE_MLThre uses an offline teacher-search procedure to construct supervised training targets for the residual threshold selector. The underlying design principle is that the bit-flipping threshold should be treated as a state-dependent decoding-control variable rather than as a globally fixed constant. In iterative decoding, the effectiveness of a threshold depends on the current residual syndrome, the progress of previous iterations, and the distribution of unsatisfied parity-check counters. Therefore, BIKE_MLThre formulates threshold selection as a local decision problem over decoder states.

For a decoder state observed at iteration (i), the offline search first constructs the same 11-dimensional feature vector used by the online inference module:

$$x_i = (level, i, S_{init}, S_{prev,i}, S_i, \Delta S_i, z_{0,i}, z_{1,i}, T_{var,i}, T_{lb}, T_{base,i})$$

For each security category, BIKE_MLThre defines a security-specific residual action set D_{sec} . In the current implementation, the candidate correction sets are $\{-7, -6, -5, -4, -3, -2, -1, 0\}$, $\{-10, -9, -8, -7, -6, -5, -4, -3, -2, -1, 0\}$ and $\{-2, -1, 0, 1, 2\}$.

For each candidate threshold $\tau_{i,d}$, the remaining decoding process is rolled out from the same recorded decoder state. The candidate producing the best terminal decoding behavior is selected as the teacher action. Let $\mathcal{L}(\tau_{i,d}; x_i)$ denote the rollout loss obtained by applying the candidate threshold $\tau_{i,d}$ at state x_i . A typical rollout loss is defined as

$$\alpha \cdot \mathbf{1}_{fail} + \beta \cdot wt(s_{final}) + \gamma \cdot wt(e_{res}) + \eta \cdot N_{flip}$$

where ($\mathbf{1}_{fail}$) indicates final decoding failure, (s_{final}) is the final syndrome, (e_{res}) is the residual error estimate after decoding, and (N_{flip}) is the number of bit flips performed during the rollout. The coefficients α, β, γ , and η control the relative importance of these evaluation terms.

The teacher residual label is defined as the residual action that minimizes the rollout loss:

$$\arg \min_{d \in D_{sec}} \mathcal{L}(clip(T_{base}, i+d, T_{min}, T_{max}); x_i)$$

If multiple residual actions achieve the same minimum loss, a fixed deterministic tie-breaking rule may be applied, such as selecting the correction with the smallest absolute value.

The learning objective is to train a compact classifier f_θ that maps the 11-dimensional decoder-state feature vector x_i to the teacher residual action d_i^*

. The model output is a classification score vector over the security-specific action set:

$$z_i = f_\theta(x_i)$$

with output dimension

$$|D_{sec}|$$

During online decapsulation, the predicted residual correction is selected by

$$Dsec \left[\arg \max_k z_{i,k} \right]$$

and the effective threshold used by the BGF decoder is computed as

$$T_i = clip(T_{base,i} + \Delta T_i, T_{min}, T_{max}).$$

The offline training procedure is specified as follows.

Offline-Teacher-Search()

1. For each security category $level \in 128, 256, 512$, generate random secret keys and random weight-(t) error patterns.
2. Execute the reference BGF decoding process and record the 11-dimensional decoder-state feature vector at each iteration:

$$x_i = (level, i, S_{init}, S_{prev,i}, S_i, \Delta S_i, z_{0,i}, z_{1,i}, T_{var,i}, T_{lb}, T_{base,i})$$

3. Select the corresponding residual action set ($\mathcal{D}_{\text{sec}}$) according to the security category.

4. For each recorded state x_i , enumerate the candidate threshold set

$$T_i = clip(T_{base,i} + \Delta T_i, T_{min}, T_{max}).$$

5. For each candidate threshold, roll out the remaining decoding iterations from the same recorded state.

6. Compute the rollout loss for each candidate and select the teacher residual label

$$\arg \min_{d \in Dsec} \mathcal{L}(clip(T_{base,i} + d, T_{min}, T_{max}); x_i)$$

7. Train f_θ to predict d_i^* from (x_i) .

8. Quantize the trained model and fix the model parameters, quantization format, feature-normalization constants, residual action set $Dsec$, confidence threshold, and admissible threshold interval $[T_{min}, T_{max}]$ as part of the algorithm instance used by *Decaps*.

This training procedure moves the expensive threshold search entirely to an offline phase. During online decapsulation, BIKE_MLThre only evaluates the fixed low-cost inference function f_θ and applies the predicted residual correction to the baseline threshold. Since the offline teacher-search procedure and the online decoder use the same 11-dimensional feature representation, the same security-category definition, and the same residual action set D_{sec} , the learned threshold-control mechanism remains aligned with the deployed implementation.

The offline search procedure is not executed during decapsulation and does not introduce runtime adaptivity beyond the fixed inference rule specified by f_θ . All deployed model parameters and threshold-control constants shall be fixed before deployment.

2 Design Rationale

2.1 Mathematical hard problems

The security of BIKE_MLThre is based on the hardness of decoding problems over quasi-cyclic moderate-density parity-check codes. For the selected parameter categories, the two underlying computational problems are quasi-cyclic syndrome decoding and quasi-cyclic codeword finding.

At the public-key level, an adversary is given an element: $h = h_1 h_0^{-1} \in R_r$, where $((h_0, h_1))$ is a sparse secret pair sampled from

$$\mathcal{H}_w = \{(h_0, h_1) \in R_r^2 : wt(h_0) = wt(h_1) = \frac{w}{2}\}.$$

Recovering an equivalent sparse secret representation from (h) corresponds to the quasi-cyclic codeword finding problem. This problem captures the difficulty of deriving the secret key from the public key.

For ciphertext decoding, the adversary is given a syndrome-like relation of the form $c_0 = e_0 + e_1 h \in R_r$

where the error pair (e_0, e_1) is sampled from

$$E_t = \{(e_0, e_1) \in R_r^2 : wt(e_0) + wt(e_1) = t\}.$$

Recovering (e_0, e_1) from (h, c_0) corresponds to the quasi-cyclic syndrome decoding problem. This problem captures the difficulty of recovering the encapsulated error vector and, consequently, the encapsulated seed.

The learning-assisted threshold selector in BIKE_MLThre is not introduced as an additional hardness assumption. It is an internal decoding-control component used during decapsulation after the secret sparse parity-check structure is available. Its role is to select an effective bit-flipping threshold from decoder-side state information, thereby improving decoding reliability. The selector does not define a new public algebraic problem, does not hide any secret material, and is not relied upon for computational secrecy.

Therefore, the mathematical security claim of BIKE_MLThre is tied to the QCSD and QCCF assumptions for the selected parameter sets. The learning-assisted component affects the decoder failure behavior and implementation profile, so its validation requirements concern decoder failure rate, constant-time execution, bounded threshold deviation, and robustness of the fixed inference rule rather than the hardness of the underlying code-based problems.

2.2 Main design strategy

The main design strategy of BIKE_MLThre is to formulate threshold selection in bit-flipping decoding as a state-dependent control problem. In BGF-style iterative decoding, the effectiveness of a flipping threshold depends on the residual syndrome, the progress of previous iterations, and the distribution of unsatisfied parity-check counters. A threshold that is suitable at one decoding state may be suboptimal at another state, even under the same parameter category. Therefore, BIKE_MLThre replaces a purely predetermined threshold schedule with a residual threshold-control mechanism driven by decoder-state information.

The proposed strategy separates threshold optimization from online decapsulation. During the offline phase, decoding traces are collected under each parameter category. For each recorded decoder state, BIKE_MLThre evaluates a small neighborhood of candidate thresholds around the reference BGF threshold. Each candidate is tested by rollout from the same state, and the threshold that yields the best subsequent decoding behavior is selected as the teacher action. This procedure produces supervised training data for approximating the state-to-threshold mapping without requiring expensive threshold search during actual decapsulation.

During the online phase, the decoder does not enumerate candidate thresholds. Instead, it evaluates a compact fixed inference function that predicts a small residual correction to the reference threshold. For a decoder state (x_i) at iteration (i) , the effective threshold is computed as

$$T_i = \text{clip}(T_{\text{base},i} + \Delta T_i, T_{\min}, T_{\max}).$$

where $\Delta T_i = f_{\theta}(x_i)$ is the learned residual correction, and $T_{\text{ref}}(S_i, i)$ is the reference BGF threshold determined by the current syndrome weight S_i and the iteration index i . This residual form allows the learned component to refine the threshold locally while keeping the decoding trajectory constrained within a controlled threshold range.

The design is guided by the following principles.

First, the threshold policy is made adaptive at the decoder-state level. The selector uses quantities that characterize the current decoding condition, including the security category, iteration index, initial syndrome weight, previous and current syndrome weights, syndrome-weight decrease, syndrome transition features, and baseline threshold information.

Second, costly threshold optimization is moved to the offline phase. Candidate-threshold evaluation, rollout simulation, and teacher-label construction are performed during training. Online decapsulation only requires a fixed inference step and a bounded threshold adjustment.

Third, the learned policy is residual rather than fully generative. The inference module predicts a correction term relative to the reference threshold instead of replacing the decoding rule entirely. This improves interpretability, limits unstable threshold deviations, and makes the effect of the learned component easier to audit.

Fourth, the online inference path is designed for efficient and deterministic implementation. The deployed module uses fixed parameters, fixed feature normalization, fixed quantization, and a fixed execution schedule. It may be instantiated either as a small quantized neural network or as a fixed lookup table, provided that the resulting threshold-selection function is fully specified before deployment.

Finally, the adaptive component is confined to threshold selection inside the decapsulation decoder. It does not introduce a new public algebraic problem or a new security assumption. Its purpose is to improve decoding reliability by reducing the decoder failure rate, while the

underlying code-based hardness assumptions remain those associated with the selected parameter category.

Difference from BIKE

BIKE_MLThre is proposed within the QC-MDPC code-based KEM design line represented by BIKE. Its construction follows the general quasi-cyclic code setting, Niederreiter-style encapsulation structure, and QCSD/QCCF-based security analysis framework that are commonly used in BIKE-like schemes. However, BIKE_MLThre is not intended to be a simple reuse of an existing BIKE implementation or a mere parameter substitution. Its main technical modification is introduced at the decoder-policy level, specifically in the threshold-selection mechanism of the BGF-derived iterative decoder used during decapsulation.

In conventional BIKE-style decoding, the bit-flipping threshold is mainly determined by a predefined threshold rule. In BIKE_MLThre, threshold selection is reformulated as a state-dependent control problem. At each decoding iteration, the decoder extracts state features such as syndrome weight, syndrome-transition information, iteration index, and baseline threshold values. A fixed lightweight inference module then predicts a residual threshold correction T_i . The effective threshold is computed as

$$T_i = \text{clip}(T_{\text{base},i} + \Delta T_i, T_{\min}, T_{\max}).$$

Thus, the difference of BIKE_MLThre does not lie in changing the underlying code-based hard problems, but in introducing a learning-assisted residual threshold-control mechanism inside the decapsulation decoder. The learning module does not participate in key generation, error sampling, ciphertext construction, or key derivation. Instead, it acts as a reliability-oriented decoder component that refines the threshold locally according to the current residual decoding state.

From an algorithmic perspective, BIKE_MLThre remains compatible with the broader BIKE-style KEM framework, while adding a new adaptive threshold-control strategy at the decoding-policy level. Its core contribution is to extend the fixed or empirically designed threshold-selection process into a trainable, quantizable, and fixed-deployment residual decision module. This design aims to improve the convergence behavior of bit-flipping decoding and reduce decoder failure risk,

without introducing a new public algebraic problem or an additional cryptographic hardness assumption.

2.3 Parameter selection basis

The parameter selection of BIKE_MLThre consists of two layers: the cryptographic parameter layer and the adaptive threshold-control layer. The cryptographic parameter layer determines the underlying quasi-cyclic code structure, the target error weight, the secret-key sparsity, the shared-secret length, and the intended security category. The adaptive threshold-control layer determines how the decoder selects residual threshold corrections during decapsulation.

The cryptographic parameters are denoted by

$$(r, w, t, \ell)$$

where (r) is the quasi-cyclic block length, (w) is the total row weight of the sparse secret parity-check representation, (t) is the error weight used during encapsulation, and (ℓ) is the shared-secret length. These parameters determine the size of the search space for the underlying QCCF and QCSD problems, as well as the decoding-failure-rate target for each parameter category. In BIKE_MLThre, these parameters are fixed per security category and are not tuned by the learning module.

The additional parameters introduced by BIKE_MLThre are associated only with the residual threshold selector. They control the magnitude of threshold correction, the confidence-based fallback behavior, the feature representation, and the implementation form of the fixed inference rule.

The residual action set (D) is intentionally small. This limits the influence of the learned component to local threshold refinement and prevents the online policy from producing large, hard-to-audit deviations from the reference threshold. The clipping interval $([T_{\min}, T_{\max}])$ provides an additional safety constraint by ensuring that every threshold used during decoding remains within an admissible range. A reference implementation uses the following threshold-control parameters.

Table 5. Reference Configuration of the BIKE_MLThre Residual Threshold Selector

Parameter	Reference choice	Role
Residual action set D_{sec}	Security-category-specific	Defines the candidate threshold corrections for each security category.
Confidence threshold τ_{conf}	Calibrated per security category	Forces fallback to $\Delta T_i=0$ under low confidence.
Model architecture	11→16→/	quantized classifier
Calibration granularity	Per-security-category model or unified security-aware model	Supports either category-specific models or a shared model conditioned on the security category.
Threshold interval	$[T_{min}, T_{max}]$	Prevents inadmissible or unstable threshold deviations.
Feature dimension	11	Defines the decoder-state representation used by the residual threshold selector.

At decoding iteration i , the residual threshold selector outputs a correction value from the action set associated with the selected security category:

$$\Delta T_i \in D_{sec}.$$

The residual action set D_{sec} is intentionally small for each security category. This limits the influence of the learned component to local threshold refinement and prevents the online policy from producing large, hard-to-audit deviations from the baseline threshold. For the 128-bit and 256-bit categories, the candidate correction sets mainly allow negative corrections, which enables the selector to moderately reduce the baseline threshold when doing so improves the bit-flipping process. For the 512-bit category, the candidate correction set allows both negative and positive corrections, enabling the selector either to lower the threshold to increase flipping activity or to raise the threshold to avoid excessive flipping.

The confidence threshold τ_{conf} is selected by validation on independent decoding traces. If the model confidence at iteration i is below this threshold, the residual correction is suppressed:

$$\Delta T_i \leftarrow 0.$$

This fallback rule ensures that uncertain predictions do not affect the decoder trajectory. The value of τ_{conf} shall be fixed before deployment and reported together with the trained model parameters, quantization format, feature-normalization parameters, and validation results.

The feature set is selected to capture decoder-state information relevant to threshold control. In the reference implementation, the residual threshold selector uses an 11-dimensional feature vector

$$x_i = (\text{level}, i, S_{\text{init}}, S_{\text{prev}, i}, S_i, \Delta S_i, z_{0, i}, z_{1, i}, T_{\text{var}, i}, T_{\text{lb}}, T_{\text{base}, i})$$

The reference inference module is a small quantized classifier with 11 input features, one hidden layer of width 16, and $|D_{\text{sec}}|$ output classes corresponding to the residual action set of the selected security category. Equivalently, the inference function can be written as

$$f_{\theta}: \mathbb{R}^{11} \rightarrow \mathbb{R}^{|D_{\text{sec}}|}.$$

This parameterization reflects the central design goal of BIKE_MLThre: the algebraic security parameters define the underlying code-based hardness level, while the additional threshold-control parameters regulate only the reliability-oriented behavior of the decapsulation decoder. The learning module is therefore used to improve threshold selection and reduce decoder failures, not to introduce a new security assumption or a new public mathematical problem.

2.4 Failure probability analysis and impact on security

The decapsulation decoder may fail with a small but non-zero probability. In BIKE_MLThre, the learning-assisted threshold selector is introduced to reduce the decoder failure rate by improving the threshold decision made during iterative bit-flipping decoding. The relevant failure event is the inability of the decoder to recover the encapsulated error vector from the syndrome determined by the secret sparse parity-check representation.

For a security category $sec \in 128, 256, 512$, with secret-key space ($\mathcal{H}_w$) and error space (E_t), the decoder failure rate is defined as

$$\Pr_{(h_0, h_1) \leftarrow \mathcal{H}_w; (e_0, e_1) \leftarrow E_t} [\text{Decoder}(e_0 h_0 + e_1 h_1; h_0, h_1) \neq (e_0, e_1)] .$$

Equivalently, (DFR) measures the probability that decapsulation fails to reconstruct the error vector sampled during encapsulation. This probability is evaluated over randomly generated secret keys and uniformly sampled weight- (t) error vectors. In implementations that distinguish between average-case and key-dependent behavior, both the average decoder failure rate and the distribution of per-key failure rates should be reported.

A low decoder failure rate is required for correctness and is also relevant to the security analysis of BIKE-like KEMs using implicit rejection. If the decoder fails, the decapsulation procedure derives the output key from the secret fallback value rather than from the recovered encapsulation seed. This prevents direct exposure of the failure event at the KEM interface, but it does not eliminate the need to control the underlying failure probability. Excessive or highly key-dependent failure behavior may increase the relevance of reaction-style attacks, weak-key phenomena, or implementation-level distinguishers. Therefore, BIKE_MLThre treats DFR reduction as a reliability and failure-resilience objective, not as a replacement for the underlying code-based hardness assumptions.

The learning-assisted threshold selector is designed to affect the decoder failure behavior in a controlled manner. For the selected security category sec , the residual correction is restricted to a security-category-specific action set.

The effective threshold is constrained by $T_i = \text{clip}(T_{\text{base}, i} + \Delta T_i, T_{\min}, T_{\max})$.

A confidence-gated fallback mechanism is used to suppress uncertain predictions. If the model confidence c_i at iteration i satisfies $c_i < \tau_{\text{conf}}$, then the residual correction is set to $\Delta T_i \leftarrow 0$.

The number of decoding iterations remains fixed, and the inference module is executed according to a fixed schedule. These restrictions are intended to prevent the adaptive threshold policy from introducing uncontrolled decoding behavior or secret-dependent execution patterns.

Table 6. Effect of ML-based Threshold Correction on Iterative Decoding Failure Rate

Instance	Iteration	Baseline Failure Rate	BIKE_MLThre Failure Rate
----------	-----------	-----------------------	--------------------------

128	1	100%	100%
128	2	29.274%	0%
128	3/4/5	0%	0%
256	1	100%	100%
256	2	97.111%	0%
256	3/4/5	0%	0%
512	1	100%	100%
512	2	44.533%	0%
512	3/4/5	0%	0%

Table 6 compares the iteration-wise decoding failure rates of the baseline decoder and BIKE_MLThre under the 128-bit, 256-bit, and 512-bit security categories. The results show that both methods fail after the first iteration, indicating that a single bit-flipping iteration is insufficient to complete decoding for all tested categories. However, a clear difference appears in the second iteration. The baseline decoder still has non-negligible failure rates, namely 29.274% for the 128-bit category, 97.111% for the 256-bit category, and 44.533% for the 512-bit category. In contrast, BIKE_MLThre reduces the failure rate to 0% at the second iteration for all three categories.

This result the main advantage of the ML-based residual threshold selector. By adapting the threshold according to the current decoder state demonstrates, BIKE_MLThre improves the intermediate convergence behavior of the iterative bit-flipping decoder. The learned threshold correction helps the decoder reach successful decoding earlier, instead of relying on later iterations to eliminate the remaining residual syndrome.

For iterations 3, 4, and 5, both the baseline decoder and BIKE_MLThre achieve 0% failure rate. Therefore, the table should not be interpreted as showing a difference in final decoding success under the tested setting. Rather, it shows that BIKE_MLThre accelerates convergence and reduces intermediate decoding failures. Since the target DFR of BIKE-like schemes is extremely low, the difference in final decoder failure rate is difficult to observe directly in limited

experiments. The iteration-wise failure rate therefore provides a more practical way to reveal the convergence advantage introduced by the ML-based threshold correction.

Overall, the results support the effectiveness of the ML-assisted threshold-control mechanism. The learned residual correction does not change the underlying code-based problem, but it improves the decoder's state-dependent threshold decision, leading to earlier failure elimination and better iterative decoding behavior.

2.5 Other considerations

BIKE_MLThre is designed with deployment compatibility, implementation efficiency, modularity, and auditability in mind. These considerations are important because the proposed learning-assisted component is placed inside the decapsulation decoder and must therefore be specified in a way that is deterministic, reproducible, and suitable for cryptographic implementation review.

****Compatibility.****

BIKE_MLThre uses the same external KEM object types as the selected BIKE parameter category, including the public key, secret key, ciphertext, and shared-secret output. Interoperability therefore depends on both parties selecting the same decoder profile and the same fixed threshold-control instance. In practice, the parameter category, decoder profile, trained model identifier, quantization format, confidence threshold, and clipping interval should be treated as part of the negotiated or specified algorithm instance.

****Implementability.****

The adaptive threshold selector relies only on decoder-side quantities that are naturally produced during BGF-style decoding, such as syndrome weight, iteration index, black/gray set information, and UPC-counter statistics. Therefore, the additional online cost is limited to feature extraction, fixed inference, confidence evaluation, and bounded threshold correction. A reference implementation should use fixed-size integer arithmetic, fixed memory access patterns where applicable, and a fixed inference schedule.

****Modularity.****

The threshold-control component is separated from the algebraic KEM layer. The trained model, feature normalization constants, residual action set, confidence threshold, and clipping bounds form a decoder sub-profile. This sub-profile may be recalibrated for different parameter categories or implementation constraints, provided that the resulting inference rule is fixed before deployment and is validated under the required DFR and side-channel criteria.

****Auditability.****

The residual design makes the effect of the learning module directly inspectable. At each iteration, the effective threshold is expressed as $T_i = \text{clip}(T_{\text{base},i} + \Delta T_i, T_{\text{min}}, T_{\text{max}})$.

where (ΔT_i) is selected from a small finite action set. This allows implementers and evaluators to compare the adaptive decoder against the reference threshold policy on a per-iteration basis. The confidence-gated fallback rule further improves auditability by defining a deterministic condition under which the learned correction is suppressed.

Overall, these considerations ensure that the learning-assisted threshold selector remains a controlled decoder component rather than an open-ended adaptive mechanism. Its parameters, execution path, fallback behavior, and validation requirements are fixed as part of the algorithm instance, enabling reproducible testing and independent implementation review.

3 Security Statement and Analysis

3.1 Security statement

BIKE_MLThre is intended to achieve code-based KEM security for the security categories $sec \in 128, 256, 512$. Its security claim is based on the hardness of the underlying quasi-cyclic code problems defined by the selected parameter category, namely quasi-cyclic syndrome decoding and quasi-cyclic codeword finding. The learning-assisted component is not used as an additional cryptographic hardness assumption; it is an internal decapsulation mechanism for controlling the threshold used by the iterative bit-flipping decoder.

At the public-algebraic level, an adversary is given the public key and ciphertext relation associated with the selected quasi-cyclic code instance. Breaking the scheme requires either recovering an equivalent sparse secret representation, solving the corresponding decoding problem for the encapsulated error vector, or distinguishing the derived shared secret from random under

the specified KEM security model. The adaptive threshold selector does not define a new public problem and does not provide secrecy by itself.

At the KEM level, BIKE_MLThre targets IND-CPA and IND-CCA security under the standard code-based assumptions and the correctness requirements of the decapsulation procedure. In particular, the IND-CCA claim relies on the implicit-rejection mechanism, the key-derivation procedure, and a sufficiently low decoder failure probability. Since BIKE_MLThre modifies the threshold-control rule used during decapsulation, the decoder correctness and failure behavior must be re-established for the specified learned threshold policy.

For a selected security category sec , the decoder failure rate is denoted by DFR_{sec} .

The intended correctness requirement is that DFR_{sec} remains sufficiently small for the target security category and for the intended deployment profile. In particular, the learned threshold selector must not introduce excessive average-case failures, highly key-dependent failure behavior, or observable failure patterns that could weaken the implicit-rejection mechanism.

The residual threshold selector is fixed as part of the algorithm instance. The online decapsulation procedure does not perform threshold search or update the model.

The security validation of BIKE_MLThre consists of the following obligations.

1. The algebraic security of each security category shall be evaluated with respect to known attacks against quasi-cyclic syndrome decoding and quasi-cyclic codeword finding.
2. The decoder failure rate shall be measured for the learned threshold policy using large-scale independent simulations. The evaluation shall include average-case DFR, per-key failure behavior, and tail-risk analysis.
3. The residual threshold selector shall be fixed before deployment, including model weights, quantization parameters, feature-normalization constants, confidence threshold, residual action set D_{sec} , and clipping interval $[T_{min}, T_{max}]$.
4. The threshold-control module shall be implemented with a fixed execution schedule and without secret-dependent branches, memory accesses, or timing variation that could expose decoder-state information.

5. The validation of the specified decoder family shall account for the learned threshold controller, including its effect on correctness, failure probability, implicit rejection, and potential reaction-style leakage.

3.2 Theoretical security analysis

BIKE_MLThre is analyzed as a Niederreiter-style QC-MDPC key encapsulation mechanism with implicit rejection in the Fujisaki-Okamoto/HHK transform framework. The public-key encryption layer is based on quasi-cyclic moderate-density parity-check codes, while the KEM layer derives the shared secret through hash-based key derivation and a decapsulation verification step.

The security analysis follows the standard proof framework used for BIKE. The adaptive threshold selector is treated as part of the deterministic decapsulation decoder. Once the decoder, its trained parameters, quantization constants, confidence threshold, residual action set, and clipping interval are fixed, it defines a concrete decoder family Dec_{MLThre} .

The threshold selector is therefore not modeled as an additional cryptographic oracle or as an independent hardness assumption. Its effect is captured through the correctness term, namely the decoder failure rate of (Dec_{MLThre}) .

The theoretical model is based on the following assumptions.

Assumption A1. The quasi-cyclic syndrome decoding problem at the selected parameter category is hard on average.

Assumption A2. The quasi-cyclic codeword finding problem at the selected parameter category is hard on average.

Assumption A3. The decoder family (Dec_{MLThre}) is $(\delta_{MLThre,sec})$ -correct, $sec \in \{128, 256, 512\}$, meaning that its average decoder failure rate satisfies the required correctness bound for the intended security category.

Assumption A4. The deployed threshold-control module is implemented with fixed-time, side-channel-resistant execution, including fixed control flow, fixed memory-access structure where applicable, and fixed inference latency.

Under Assumptions A1--A4, BIKE_MLThre satisfies the corresponding IND-CPA and IND-CCA security statements in the random-oracle model, with an additional correctness term determined by $\text{DFR}(\text{Dec}_{\text{MLThre}})$.

The central validation obligation introduced by BIKE_MLThre is therefore the establishment of a sufficiently small value of (δ_{MLThre}) for the specified learned threshold policy.

3.2.1 Theoretical Security Analysis

This subsection states the security-reduction skeleton for BIKE_MLThre. The purpose is to identify the algebraic problems, the correctness condition, and the role of the adaptive decoder in the security argument.

Let $(\text{Dec}_{\text{MLThre}})$ denote the BGF-derived decoder whose threshold at iteration (i) is computed as

$$T_i = \text{clip}(T_{\text{base},i} + \Delta T_i, T_{\min}, T_{\max})$$

where $\Delta T_i = D_{\text{sec}} \left[\arg \max_k z_{i,k} \right]$, with confidence-gated fallback to $(\Delta T_i = 0)$ when required.

Algebraic correctness identity

Let $h = h_1 h_0^{-1}$ in R be the public key element, and let $e = (e_0, e_1)$ in E_t be the error vector sampled during encapsulation. The ciphertext syndrome component is $c_0 = e_0 + e_1 h$ satisfies

$$c_0 h_0 = (e_0 + e_1 h_1 h_0^{-1}) h_0 = e_0 h_0 + e_1 h_1.$$

Thus, decapsulation reduces to recovering $((e_0, e_1))$ from the QC-MDPC syndrome $s = e_0 h_0 + e_1 h_1$.

BIKE_MLThre uses $(\text{Dec}_{\text{MLThre}})$ to solve this syndrome-decoding instance. The adaptive threshold policy affects the decoding trajectory and the failure probability, while the syndrome equation and the algebraic decoding problem remain those specified above.

Underlying Public-Key Encryption Scheme

Define the underlying Niederreiter-style encryption scheme ($PKE0_{MLThre}$) as follows.

KeyGen0: sample $(h_0, h_1) \leftarrow H_w$ and set $h \leftarrow h_1 h_0^{-1}$.

Encrypt0: on input h and $e = (e_0, e_1)$ in E_t , output $c_0 \leftarrow -e_0 + e_1 h$.

Decrypt0: on input (h_0, h_1) and c_0 , output $Dec_{MLThre}(c_0 h_0, h_0, h_1, sec)$.

The public challenge in ($PKE0_{MLThre}$) is the quasi-cyclic syndrome relation ($s = e_0 + e_1 h$), while the decryption algorithm invokes the specified adaptive decoder.

Theorem 1: OW-CPA inheritance from QCSD and QCCF.

For any OW-CPA adversary A_0 against $PKE0_{MLThre}$, there exists a distinguisher D against QCCF, running in about the same time, such that

$$Adv_{OW-CPA}^{PKE0-MLThre}(A_0) \leq Adv_{IND}^{QCCF}(D) + Adv_{OW}^{QCSD}(A_0).$$

Proof sketch. The proof proceeds through the standard game sequence for quasi-cyclic Niederreiter encryption. In the real OW-CPA game, the public key is $h = h_1 h_0^{-1}$ for (h_0, h_1) sampled from H_w . In the hybrid game, h is replaced by a uniformly random odd element of R . Distinguishing the real public-key distribution from the hybrid distribution corresponds to the QCCF distinguishing problem. In the hybrid game, recovering e from $s = e_0 + e_1 h$ is exactly the QCSD search problem. The adaptive decoder is not part of the encryption-side challenge distribution; its role appears in the correctness analysis of decryption.

Lemma 1: IND-CPA Security of the Randomized PKE

Let PKE-MLThre be the randomized encryption scheme obtained from $PKE0_{MLThre}$ by encrypting a message m as

$$c = (c_0, c_1) = (e_0 + e_1 h, m \text{ xor } L(e_0, e_1)), \text{ where } (e_0, e_1) \leftarrow E_t.$$

Then for any IND-CPA adversary A against PKE-MLThre, there exists an OW-CPA adversary A_0 against $PKE0_{MLThre}$, running in about the same time, such that

$$Adv_{IND-CPA}^{PKE-MLThre}(A) \leq (1/2) Adv_{OW-CPA}^{PKE0-MLThre}(A_0).$$

Combining this lemma with Theorem 1 yields

$$Adv_{IND-CPA}^{PKE-MLThre}(A) \leq (1/2)Adv_{IND}^{QCCF}(D) + (1/2)Adv_O^{QCSD}(A_0).$$

Proof sketch. The one-time-pad component $c_1 = m \text{ xor } L(e)$ hides the challenge bit unless the adversary queries the random oracle L at a valid witness e for the challenge syndrome c_0 . Such a query immediately gives an OW-CPA solution to PKE0-MLThre. This is the standard BIKE argument from randomized Niederreiter-style encryption to IND-CPA security.

Delta-Correctness of the Adaptive Decoder.

For the IND-CCA analysis, the relevant correctness quantity is the average failure probability of the deployed decoder:

$$\delta_{MLThre} = DFR(Dec_{MLThre}) = Pr[Dec_{MLThre}(e_0 h_0 + e_1 h_1, h_0, h_1) \neq (e_0, e_1)],$$

The probability is taken over the specified key-generation distribution and the specified weight-(t) error distribution. Since the syndrome depends on the error vector and the public code instance, the correctness event is independent of the protected message value. Therefore, the randomized PKE is (δ_{MLThre}) -correct whenever the above failure probability is bounded by (δ_{MLThre}) .

Theorem 2: inherited IND-CCA statement for BIKE_MLThre.

Assume that the hash functions used for error sampling, masking, and key derivation are modeled as random oracles, and assume that Dec_{MLThre} is (δ_{MLThre}) -correct. For any IND-CCA adversary $\mathcal{B}$ against BIKE_MLThre issuing at most q , denotes the total number of relevant random-oracle and decapsulation-related queries. there exist a QCCF distinguisher D and a QCSD solver S , running in comparable time, such that

$$Adv_{IND-CCA}^{BIKE_{MLThre}}(B) \leq q * \delta_{MLThre} + 3q/|M| + (3/2)Adv_{IND}^{QCCF}(D) + (3/2)Adv_O^{QCSD}(A_0).$$

Proof sketch. The proof applies the HHK/FO-style implicit-rejection analysis to the randomized Niederreiter encryption scheme described above. Decapsulation reconstructs a candidate seed, recomputes the corresponding error vector, verifies the ciphertext consistency

condition, and derives the shared secret either from the recovered seed or from the secret fallback value. The correctness loss contributes the term $q\delta_{MLThre}$, and the random-oracle guessing loss contributes the term $\frac{3q}{|\mathcal{M}|}$. The remaining advantage is bounded by the IND-CPA security of the underlying randomized PKE. Substituting the IND-CPA bound gives the stated inequality.

Concrete consequence for parameter selection.

For a target classical security level sec , the following conditions are required.

1. $QCCF_{r,w}$ offers at least λ bits of security.
2. $QCSD_{r,t}$ offers at least λ bits of security.
3. $M = 2^l \geq 2^{sec}$.
4. $(Dec_{MLThre}) \leq 2^{-sec}$ for the selected parameter set and implementation model.
5. The complete decapsulation path, including the learned threshold selector, confidence gate, clipping operation, and failure-handling logic, is implemented in fixed time and does not expose decoder-state information through timing, branching, memory access, or failure behavior.

The first three conditions concern the algebraic security level and the message-space size. The fourth condition is the main correctness requirement for the adaptive decoder. The fifth condition is an implementation-security requirement that prevents the threshold-control module from becoming a side-channel oracle.

3.3 Concrete security analysis

For all three instances, the dominant known attack families remain the same as for BIKE:

- Information-set decoding and its quasi-cyclic optimizations against QCSD and related search/decision variants.
- Codeword-finding attacks exploiting the quasi-cyclic structure.
- Reaction-style and weak-key attacks that become relevant if DFR is too high.
- Quantum speedups affecting the complexity of code-based attacks, as already considered in the BIKE parameter selection philosophy.

Since BIKE_MLThre keeps the same (r,w,t) values as BIKE, the best-known attack surface at the algebraic level is unchanged. The concrete security objective is therefore to retain BIKE's NIST-style categories while improving decoder correctness.

Table 7.DFR for BIKE_MLThre

Instance	Classical target	Quantum target	Dominant known attacks	DFR
128	Above 128bit	Above 80bit	ISD / QCSD / QCCF	2^{-128}
256	Above 256bit	Above 128bit	ISD / QCSD / QCCF	2^{-256}
512	Above 512bit	Above 256bit	ISD / QCSD / QCCF	2^{-512}

The listed work-factor targets describe the intended classical security strength associated with the selected parameter categories. Quantum attacks are considered through the security-category methodology used for code-based parameter assessment, including the effect of quantum speedups on the best known decoding and codeword-finding attacks.

3.4 Other security properties

Side-channel resistance

The complete decapsulation path, including the adaptive threshold selector, shall be implemented in a side-channel-resistant manner. The threshold module must use fixed-size arithmetic, fixed control flow, and fixed memory-access patterns where applicable. It shall not introduce secret-dependent branches, variable-time table lookups, early termination based on secret-dependent decoder states, or data-dependent model execution paths.

A preferred deployment form is a small quantized inference module or a fixed lookup table evaluated under a constant-time execution schedule. The model weights, lookup-table entries, quantization parameters, feature-normalization constants, confidence threshold, residual action set, and clipping interval shall be fixed before deployment. The confidence gate and clipping operation shall also be implemented without secret-dependent timing variation.

Forward secrecy.

Forward secrecy is a property of the surrounding key-exchange protocol rather than of the KEM algorithm alone. When BIKE_MLThre is used with ephemeral key pairs in an authenticated key-exchange protocol, the resulting deployment can support forward-secrecy-oriented protocol designs. In contrast, static-key deployments do not provide forward secrecy by themselves and

require stricter attention to decoder failure behavior, implicit rejection, side-channel resistance, and key-reuse exposure.

Multi-key and key-reuse considerations

In multi-key settings, an adversary may observe many public keys, ciphertexts, and decapsulation outcomes across different users or sessions. The relevant security concerns include the algebraic hardness of the selected parameter category, the distribution of public keys, and the decoder failure behavior across keys. For BIKE_MLThre, the adaptive threshold selector must therefore be validated not only in terms of average DFR, but also in terms of the per-key failure distribution.

Robust fallback

The confidence-gated residual policy is a risk-control mechanism for the adaptive decoder. At iteration (i), if the model confidence score (c_i) is below the configured threshold

$$c_i < \tau_{\text{conf}},$$

the residual correction is suppressed by setting

$$\Delta T_i \leftarrow 0.$$

The effective threshold is then computed from the reference threshold rule rather than from an uncertain learned correction. This fallback behavior prevents low-confidence or out-of-distribution model outputs from producing uncontrolled threshold deviations.

4 Performance Evaluation

4.1 Performance analysis

The performance profile of BIKE_MLThre consists of two parts: the external KEM cost and the internal adaptive-decoding cost. The external KEM cost is determined by the selected security category, including the public-key size, secret-key size, ciphertext size, and shared-secret size. The adaptive threshold selector is executed only inside decapsulation and therefore affects the online computational cost of the decoder rather than the communication bandwidth or the algebraic key and ciphertext formats.

The additional decapsulation cost comes from four operations: feature extraction, fixed inference, confidence evaluation, and threshold post-processing.

Most quantities used in x_i , such as syndrome weights, iteration index, threshold values, and syndrome-transition statistics, are already available or can be computed with lightweight aggregation during the BGF-style decoding process. Therefore, the additional feature-extraction cost is mainly associated with maintaining previous syndrome information, computing syndrome-weight variation, extracting the transition features $z_{0,i}$ and $z_{1,i}$, and collecting the baseline threshold values used by the decoder. The reference expectation is that the additional inference cost is small compared with the parity-check and UPC-counter processing cost of the decoder.

4.2 Performance testing methodology

The purpose of the performance evaluation is to quantify both the standard KEM operation cost and the additional online cost introduced by the adaptive threshold-control module.

Table 8 reports the computation costs of BIKE_MLThre under the AVX512 implementation profile. The results show that the computational cost increases with the target security level, as larger parameters lead to more expensive polynomial operations, syndrome processing, and iterative decoding. Among the three KEM operations, decapsulation is consistently the dominant cost. This is expected because Decaps includes the full bit-flipping decoding procedure, whereas KeyGen and Encaps mainly involve key construction, error sampling, and ciphertext generation.

For the 128-bit and 256-bit categories, the measured costs remain within a practical range for software implementation. The increase from 128-bit to 256-bit mainly reflects the larger quasi-cyclic block length and higher decoding complexity. The 512-bit category introduces a much higher computational cost, especially in Decaps, which indicates that this parameter set is more suitable for high-assurance scenarios where very high security strength is prioritized over latency.

These results also support the design choice of localizing the learning-assisted module inside the decapsulation decoder. Since the dominant cost already comes from the BGF-style decoding process, the additional cost of the residual threshold selector is expected to be small compared with the overall Decaps cost.

Overall, the performance profile shows that BIKE_MLThre preserves the standard KEM operation structure while adding only a localized adaptive decision step inside Decaps. This makes

the design suitable for implementations where a small increase in decoder-side computation can be traded for improved reliability and lower failure probability.

Table 8. Computation Costs of BIKE_MLThre

Reference BIKE software	KeyGen	Encaps	Decaps
128, AVX512	$538 \cdot 10^3 \text{cycles}$	$101 \cdot 10^3 \text{cycles}$	$1273 \cdot 10^3 \text{cycles}$
256, AVX512	$4181 \cdot 10^3 \text{cycles}$	$586 \cdot 10^3 \text{cycles}$	$13298 \cdot 10^3 \text{cycles}$
512, AVX512	$46896 \cdot 10^3 \text{cycles}$	$3649 \cdot 10^3 \text{cycles}$	$1194361 \cdot 10^3 \text{cycles}$

The implementation was evaluated on a 64-bit desktop platform equipped with an Intel Core i5-10400F CPU at 2.90 GHz, 16 GB RAM, and Windows 11 Pro. The tested software implementation includes the baseline BIKE-style decoder and the BIKE_MLThre adaptive decoder under the same code base and measurement procedure.

4.3 Transmission and storage overhead

The transmission and key-storage cost of BIKE_MLThre is determined by the selected parameter category. The public key, secret key, ciphertext, and shared-secret output are specified independently of the adaptive threshold selector. The learning-assisted component is a local decoder-profile artifact used during decapsulation and is not transmitted as part of the ciphertext.

Table 9. Storage Costs of BIKE_MLThre

Instance	Private key	Public key	Ciphertext	Extra model storage
128	2,244 bits	12,323 bits	12,579 bits	implementation-dependent; sub-kilobyte target
256	4,640 bits	40,973 bits	41,229 bits	implementation-dependent; sub-kilobyte target
512	10,340 bits	150,001 bits	150513 bits	implementation-dependent; sub-kilobyte target

Let r_{sec} denote the quasi-cyclic block length for the selected security category, and let μ_{sec} denote the bit length of the encapsulation seed or masked seed component. The ciphertext size is

$$|c| = |c_0| + |c_1| = r_{sec} + \mu_{sec}$$

bits, where $(c_0 \in R_r)$ and $(c_1 \in \{0,1\}^{\mu_{sec}})$. The public-key size is r bits, and the secret-key size is determined by the compact representation of the sparse secret pair and the fallback secret value used by decapsulation.

The following table summarizes the protocol-level transmission and storage costs. The 128-bit and 256-bit entries use the corresponding BIKE reference parameter sizes as baseline values. The 512-bit entry requires an independently specified parameter set and must be reported with the final selected parameters.

The additional storage introduced by BIKE_MLThre belongs to the decoder profile rather than to the transmitted KEM objects. This profile includes the fixed model weights or lookup table, quantization constants, feature-normalization parameters, confidence threshold, residual action set, and clipping bounds. These values are fixed before deployment and are treated as part of the algorithm instance.

With compact integer quantization, these model parameters require only a small amount of local decoder-profile storage. Additional metadata, including quantization scales, feature-normalization constants, the confidence threshold τ_{conf} , the residual action set D_{sec} , and the clipping interval $[T_{\min}, T_{\max}]$, shall also be included in the decoder profile.

Therefore, the communication overhead of BIKE_MLThre is unchanged at the KEM object level: the public key, secret key, ciphertext, and shared-secret output are determined by the selected algebraic parameters and are not enlarged by the adaptive threshold selector. The additional overhead is local to the decapsulation implementation and consists only of the fixed decoder profile used by the residual threshold selector. This distinction should be reported explicitly in implementation results: key and ciphertext sizes measure protocol-level storage and transmission cost, whereas model or lookup-table size measures local implementation overhead.

5 Feature Statement

5.1 Innovation

The main innovation of BIKE_MLThre is the introduction of a learning-assisted residual threshold controller into the decapsulation decoder. Threshold selection is formulated as a state-dependent control problem over the iterative bit-flipping process. At each decoding iteration, the

controller observes compact decoder-state features and predicts a bounded residual correction to the baseline threshold.

This design integrates machine learning at the decoder-policy level. The learning module does not participate in public-key generation, error sampling, ciphertext construction, or key derivation. Its role is limited to reliability-oriented threshold refinement during decapsulation. As a result, BIKE_MLThre uses learning to improve decoder behavior while keeping the underlying code-based security problem separate from the learned component.

5.2 Simplicity

BIKE_MLThre is structurally simple because the adaptive mechanism is localized to a single step: threshold computation inside the BGF-derived decoder. The decoder still follows a fixed iterative structure. The learned component only outputs the residual threshold correction ΔT_i . This residual formulation makes the behavior of the adaptive decoder easy to inspect, because every threshold can be compared directly with the corresponding reference threshold. The confidence-gated fallback mechanism further simplifies validation by defining a deterministic condition under which the learned correction is suppressed.

5.3 Flexibility

The threshold-control module can be instantiated in several compact forms, including a quantized multilayer perceptron, a fixed lookup table, or another deterministic classifier with the same input-output specification. It can be calibrated separately for each security category or trained as a unified security-aware model with an explicit security-category input.

The residual action set, confidence threshold, feature normalization constants, and clipping interval can be selected according to the implementation profile. These choices regulate the behavior of the threshold selector without modifying the algebraic parameter set. This separation allows implementers to tune the reliability-oriented decoder profile while keeping the cryptographic parameter category well defined.

5.4 Compatibility

BIKE_MLThre is designed to fit naturally into existing BIKE-style software and hardware implementations. The adaptive selector uses quantities already produced during BGF-style decoding, such as syndrome weights, iteration index, syndrome-weight variation, syndrome-transition features, and baseline threshold values. Therefore, an implementation can integrate

BIKE_MLThre by replacing the threshold-selection subroutine rather than redesigning the full KEM workflow.

Interoperability and reproducibility require implementations claiming the same algorithm instance to use the same security category, decoder profile, trained model or lookup table, quantization format, feature-normalization constants, confidence threshold, residual action set, and clipping interval.

5.5 Scalability and extensibility

The residual threshold-control framework can be extended in multiple directions. A minimal configuration may learn only the first-iteration threshold correction, while a richer configuration may learn corrections for the first two iterations or for the full fixed-length decoding schedule. For any concrete algorithm instance, the set of iterations to which the learned correction is applied shall be fixed before deployment. The same offline teacher-search procedure can also be used to generate improved training labels when additional decoding traces, weak-key examples, or implementation-specific failure cases become available.

The framework is extensible because the online inference rule remains fixed after deployment. Retraining or recalibration affects the decoder profile used in a new implementation instance, but it does not imply runtime learning or adaptive self-modification during decapsulation. This distinction is important for reproducibility and independent security evaluation.

5.6 Cross-platform implementation advantages

On desktop CPUs, the additional arithmetic introduced by the threshold selector is small compared with the parity-check, UPC-counter, and syndrome-update operations of decapsulation. For the reference quantized classifier

$$11 \rightarrow 16 \rightarrow |D_{sec}|,$$

the online inference cost is limited to a small number of fixed-point operations per decoding iteration.

On embedded or hardware-oriented platforms, the same threshold-selection function can be implemented as a tiny fixed-point neural network or as a fixed lookup table. Both forms allow predictable latency and a fixed execution schedule, provided that table access, branching, and

memory operations are implemented in a side-channel-resistant manner. The practical advantage of this design is that decoder reliability can be improved through a bounded local inference step.